GUÍA DE IMPLEMENTACIÓN Y DESPLIEGUE

HUELLITAS

React + Vite · Backend Node.js · PostgreSQL / Supabase · GitHub · Vercel

Ingeniería de Software I · Manual detallado para demostración del Hito 3

Versión: monorepositorio con carpetas frontend/ y backend/ | Plataforma de despliegue: dos proyectos de Vercel conectados al mismo GitHub

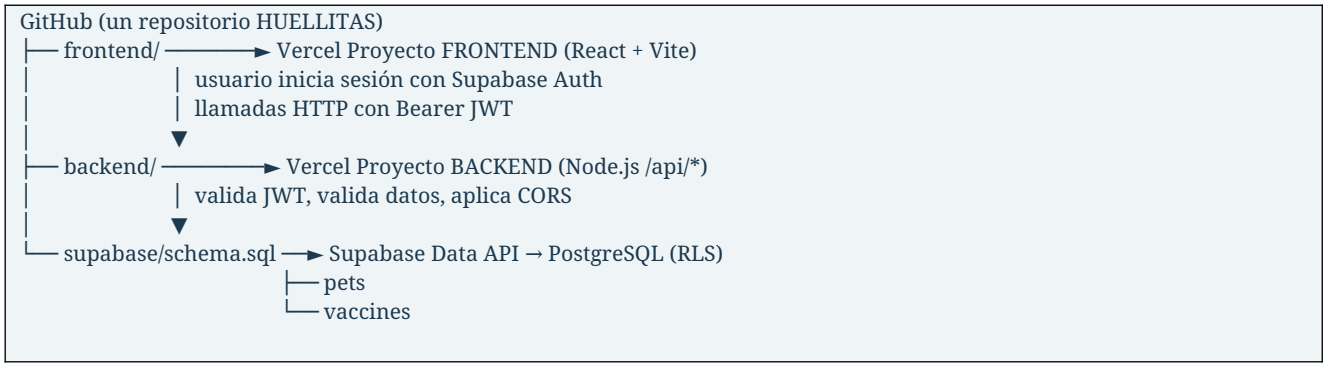
Resultado esperado — Desde un único repositorio GitHub podrás publicar una interfaz React y una API independiente en Vercel, conectadas a autenticación y PostgreSQL mediante Supabase. Solo necesitas sustituir los valores de ejemplo por los de tu cuenta.

Mapa de los pasos

Fase	Qué se configura	Comprobación
1	Supabase: proyecto, SQL, Auth y claves públicas	Tablas pets / vaccines y RLS activado
2	Backend local: API y tests	/api/health devuelve JSON y npm test pasa
3	Frontend local: Vite y login	La aplicación carga y permite crear mascotas
4	GitHub: repositorio monorepo	Se ven frontend/, backend/, supabase/
5	Vercel: proyecto Backend	API remota responde 200 / 401
6	Vercel: proyecto Frontend	Se muestra aplicación React publicada
7	CORS + Supabase Auth URLs	Login, CRUD y persistencia funcionando
8	Validación docente	Registro de pruebas y evidencia de despliegue

1. Arquitectura elegida y por qué

El código no se divide en repositorios independientes, sino en un único monorepositorio. Vercel permite importar el mismo repositorio más de una vez y asignar una carpeta raíz distinta a cada proyecto. Esta decisión facilita la trazabilidad por commits y mantiene separadas las responsabilidades de frontend, backend y persistencia.



La API aplica operaciones GET, POST, PATCH y DELETE para mascotas y vacunas. El frontend utiliza una variable VITE_API_BASE_URL para localizar el backend. El backend consulta Supabase utilizando el JWT del usuario: PostgreSQL decide a qué filas puede acceder mediante RLS.

2. Preparativos en el computador

- Instalar Node.js 22.12 o superior y npm; comprobar desde Terminal o PowerShell.
- Instalar Git y un editor (por ejemplo, Visual Studio Code).
- Crear cuentas en GitHub, Vercel y Supabase.
- Extraer Huellitas_GitHub_Frontend_Backend.zip en una carpeta de trabajo sin rutas excesivamente largas.
- Verificar que no exista ningún archivo .env.local dentro del ZIP: solo se incluyen ejemplos sin claves reales.

```
node -v
npm -v
git --version
```

3. Conocer la estructura del proyecto

Ruta	Responsabilidad
frontend/src	Vistas React, formularios, dashboard, login y cliente HTTP
frontend/vite.config.js	Servidor local React y redirección /api a backend local
backend/api	Endpoints Node.js que Vercel publica como funciones
backend/lib	Autorización JWT, CORS, respuestas y conexión con Supabase
backend/server	Servidor local y reglas de validación
backend/tests	Pruebas unitarias de validación y CORS
supabase/schema.sql	Modelo de datos, claves foráneas, restricciones, índices y RLS
README.md	Guía rápida incluida dentro del repositorio

4. Crear el proyecto PostgreSQL en Supabase

1. Ingresar a https://supabase.com/dashboard e iniciar sesión.

2. Seleccionar New project, elegir una organización, un nombre (por ejemplo huellitas-demo), una región y una contraseña de base de datos.

3. Guardar la contraseña fuera del código y fuera de GitHub. Esperar hasta que Supabase muestre el proyecto operativo.

4. Abrir SQL Editor y elegir New query.

5. Abrir supabase/schema.sql con VS Code, seleccionar su contenido completo y pegarlo en SQL Editor.

6. Presionar Run. Comprobar que no existan errores SQL y que el panel informe ejecución correcta.

7. Ir a Table Editor; verificar que existan las tablas pets y vaccines.

8. Revisar que la relación vaccines.pet_id → pets.id y las políticas RLS estén configuradas.

Atención — El archivo SQL del repositorio puede consultarse desde la raíz: supabase/schema.sql. No hace falta instalar PostgreSQL ni exponer el puerto 5432 en el computador para este ejemplo.

4.1 Verificar RLS sin debilitar la seguridad

En SQL Editor puedes ejecutar las siguientes consultas de solo lectura después de aplicar el esquema:

```
SELECT tablename, rowsecurity
FROM pg_tables
WHERE schemaname = 'public'
  AND tablename IN ('pets', 'vaccines');

SELECT tablename, policyname, cmd
FROM pg_policies
WHERE schemaname = 'public'
  AND tablename IN ('pets', 'vaccines');
```

Ambas tablas deben indicar rowsecurity = true. Deben existir políticas por operación para restringir los registros a su propietario. No desactivar RLS para solucionar un problema de permisos: diagnosticar el JWT, las políticas y el owner_id.

5. Configurar autenticación y claves públicas

1. En Supabase abrir Authentication → Providers → Email y confirmar que el inicio de sesión con correo esté habilitado.

2. Revisar la opción Confirm Email. Para una demostración con acceso real, es preferible mantener la confirmación; si se usa otra configuración en un taller cerrado, documentarla y restaurarla luego.

3. En el cuadro Connect, o en Settings → API Keys, copiar la Project URL y la Publishable key (sb_publishable_...).

4. Registrar temporalmente esos dos valores en un lugar privado para completar las variables locales y de Vercel.

Seguridad — Una clave publishable está diseñada para clientes públicos sujetos a permisos y RLS. No colocar sb_secret_..., service_role, contraseña de base de datos ni JWT en código fuente, archivos públicos, capturas o videos.

6. Ejecutar el backend local

Abre una terminal en la carpeta backend/. El repositorio contiene package.json propio del backend, por lo que debes instalar sus dependencias de manera independiente.

```
cd backend
npm install
```

Copia backend/.env.example como backend/.env.local. En Windows PowerShell puedes usar Copy-Item; en macOS/Linux, cp. Luego edita los valores.

```
# Windows (desde backend/):
Copy-Item .env.example .env.local

# macOS / Linux:
cp .env.example .env.local
```

```
SUPABASE_URL=https://TU-PROYECTO.supabase.co
SUPABASE_PUBLISHABLE_KEY=sb_publishable_TU_CLAVE
CORS_ALLOWED_ORIGINS=http://localhost:5173,http://127.0.0.1:5173
```

Ejecuta el servidor local y deja esta terminal abierta:

```
npm run dev
```

En otra pestaña, abre http://127.0.0.1:3001/api/health. El backend debería responder con JSON incluyendo "status":"ok". Abre también /api/pets sin autenticación: debería devolver HTTP 401.

6.1 Ejecutar pruebas unitarias

Desde la carpeta backend, utiliza otra terminal si el servidor está en ejecución:

```
npm test
```

Se ejecutan pruebas de validación de datos y CORS. Esta verificación no necesita cuentas de Supabase ni datos de producción. No sustituye las pruebas funcionales de extremo a extremo.

7. Ejecutar frontend React + Vite local

Abre una segunda terminal ubicada en la carpeta frontend/:

```
cd frontend
npm install
```

Copia frontend/.env.example a frontend/.env.local y completa los valores:

```
VITE_SUPABASE_URL=https://TU-PROYECTO.supabase.co
VITE_SUPABASE_PUBLISHABLE_KEY=sb_publishable_TU_CLAVE
VITE_API_BASE_URL=
```

Deja VITE_API_BASE_URL vacío para la prueba local. El servidor Vite sabe reenviar /api a http://127.0.0.1:3001. Ejecuta:

```
npm run dev
```

Abre http://localhost:5173. Registra un usuario ficticio, confirma el correo si Supabase lo requiere y luego ingresa. Prueba registrar una mascota. Si el frontend muestra la configuración pendiente, revisar los valores VITE_ y reiniciar Vite.

7.1 Compilar el frontend antes de publicar

```
# Ejecutar en frontend/
npm run build
```

El comando genera dist/. El contenido se produce de nuevo en Vercel y no debe añadirse al repositorio Git.

8. Subir el monorepositorio completo a GitHub

Crea un repositorio nuevo y vacío en https://github.com/new. No es necesario agregar README, .gitignore ni licencia desde GitHub: esos archivos ya vienen incluidos.

Abre una terminal en la carpeta principal del proyecto, donde están frontend/, backend/ y supabase/. No ejecutar git init dentro de frontend ni backend por separado.

```
git init
git add .
git status
git commit -m "Huellitas: frontend, backend y PostgreSQL"
git branch -M main
git remote add origin https://github.com/TU_USUARIO/huellitas.git
git push -u origin main
```

Si git ya tiene un repositorio, omite git init. Si ya existe el remoto, verifica con git remote -v antes de intentar agregarlo. Para autenticarte, utiliza el método recomendado por GitHub.

Control crítico — Antes del commit revisa git status: no deben aparecer archivos .env.local, ni secretos, tokens, claves de servidor, node_modules/ ni dist/. En GitHub tienen que verse las carpetas frontend/ y backend/ en el nivel superior.

9. Publicar backend en Vercel (proyecto n.° 1)

1. Ingresar en https://vercel.com/dashboard y conectar tu cuenta GitHub si todavía no lo has hecho.
2. Seleccionar Add New → Project e importar el repositorio huellitas.
3. En Root Directory pulsar Edit y elegir backend (NO backend/api).
4. Seleccionar Framework Preset: Other cuando Vercel lo solicite; mantener el comportamiento por defecto para funciones Node.js.
5. Agregar las variables SUPABASE_URL, SUPABASE_PUBLISHABLE_KEY y CORS_ALLOWED_ORIGINS.

6. En un primer despliegue, puedes dejar CORS_ALLOWED_ORIGINS solo con http://localhost:5173,http://127.0.0.1:5173; volverás a actualizarla cuando conozcas la URL del frontend.

7. Pulsar Deploy y copiar la URL asignada al proyecto API, por ejemplo https://huellitas-api.vercel.app.

8. Abrir https://huellitas-api.vercel.app/api/health y comprobar respuesta JSON con status ok.

9. Abrir /api/pets sin token y comprobar que el servidor exige autenticación (HTTP 401).

Variable Vercel backend	Valor
SUPABASE_URL	https://TU-PROYECTO.supabase.co
SUPABASE_PUBLISHABLE_KEY	sb_publishable_TU_CLAVE
CORS_ALLOWED_ORIGINS	http://localhost:5173,http://127.0.0.1:5173 (temporal)

Qué ocurre aquí — Vercel publica como funciones HTTP los archivos de backend/api/. El archivo backend/vercel.json contiene rewrites para PATCH y DELETE por ID, por ejemplo /api/pets/UUID. Supabase no ejecuta esta API personalizada: solamente proporciona autenticación, API de datos y PostgreSQL.

10. Publicar frontend en Vercel (proyecto n.° 2)

1. En Vercel seleccionar nuevamente Add New → Project.

2. Importar el mismo repositorio GitHub que usaste para el backend.

3. En Root Directory elegir frontend, para que Vercel instale el package.json de React.

4. Comprobar Framework Preset: Vite. Build Command: npm run build. Output Directory: dist.

5. Agregar VITE_SUPABASE_URL y VITE_SUPABASE_PUBLISHABLE_KEY.

6. Agregar VITE_API_BASE_URL con la URL pública exacta del backend creada en el paso 9, sin /api y sin slash final.

7. Pulsar Deploy y copiar la URL pública asignada al frontend, por ejemplo https://huellitas.vercel.app.

8. Abrir la interfaz y comprobar que aparezca el formulario de inicio de sesión.

Variable Vercel frontend	Valor
VITE_SUPABASE_URL	https://TU-PROYECTO.supabase.co
VITE_SUPABASE_PUBLISHABLE_KEY	sb_publishable_TU_CLAVE
VITE_API_BASE_URL	https://TU-BACKEND.vercel.app (sin /api)

11. Conectar frontend y backend: CORS

Como ambos proyectos Vercel tienen dominios diferentes, los navegadores aplican CORS. Debes agregar la URL del frontend a la lista permitida del backend.

1. En Vercel abrir el proyecto backend → Settings → Environment Variables.

2. Modificar CORS_ALLOWED_ORIGINS para incluir la URL real del frontend, sin slash final y separada por comas de los orígenes locales.

```
CORS_ALLOWED_ORIGINS=https://huellitas.vercel.app,http://localhost:5173,http://127.0.0.1:5173
```

3. Guardar y ejecutar Redeploy del backend para cargar la variable nueva.

4. En el navegador abrir el frontend. Si hay error, inspeccionar Network/Console y revisar la coincidencia exacta del dominio incluyendo https.

Cuidado — No desactivar CORS ni usar permisos de base de datos más amplios para resolver un error de comunicación. CORS es una restricción del navegador; los permisos de registros se controlan mediante autenticación y RLS.

12. Configurar las redirecciones de Supabase Auth

1. Entrar en Supabase → Authentication → URL Configuration.

2. Establecer Site URL con el dominio del frontend de producción: https://huellitas.vercel.app.

3. Agregar Redirect URLs para http://localhost:5173/** y https://huellitas.vercel.app/**.

4. Guardar. Crear una cuenta ficticia nueva desde el frontend de producción y revisar el correo de confirmación si está habilitado.

5. Después de confirmar, iniciar sesión. Si el mensaje de confirmación apunta a localhost, volver a revisar Site URL.

No es necesario configurar la URL del backend en Supabase Auth para este flujo: la autenticación de usuario se realiza desde el frontend, mientras que el backend comprueba el JWT.

13. Prueba extremo a extremo: demostración al docente

Código	Prueba	Resultado esperado
P01	GET /api/health en URL backend	200 con JSON status ok
P02	GET /api/pets sin token	401, autenticación requerida
P03	Registro de usuario ficticio	Cuenta creada / correo de confirmación si aplica
P04	Iniciar sesión	Panel de mascotas habilitado
P05	Agregar mascota	Mascota visible en panel y en tabla pets
P06	Editar mascota	Cambio persistente tras recargar
P07	Registrar vacuna	Vacuna enlazada al pet_id correspondiente
P08	Crear usuario B e ingresar	No puede ver mascotas del usuario A
P09	Eliminar mascota con vacunas	Mascota y vacunas asociadas desaparecen
P10	Capturar despliegues Vercel	Ambos proyectos en Ready y URLs registradas

Recomendación metodológica: para cada prueba registrar estado PASS, FAIL o BLOCKED, evidencia, fecha y commit. No afirmar que el proyecto funciona solamente porque la portada se abre.

14. Diagnóstico rápido de errores habituales

Síntoma	Causa frecuente	Solución
404 en /api/health	Se seleccionó carpeta incorrecta	Vercel backend: Root Directory = backend
500 o 503 API	Supabase env incompletas	Revisar variables backend y Redeploy
Pantalla Supabase sin configurar	Variables VITE_ faltantes	Frontend: variables correctas + Redeploy
Error CORS / preflight	Dominio frontend no autorizado	Actualizar CORS_ALLOWED_ORIGINS + Redeploy backend
401 en mascota	Sesión ausente/expirada	Reiniciar sesión y revisar JWT
403/permiso RLS	Intento de acceder a datos de otro usuario	Revisar owner_id / políticas sin desactivarlas
Confirmación apunta a localhost	Site URL incorrecta	Corregir Supabase Auth URL Configuration
El frontend sigue llamando localhost	VITE_API_BASE_URL incorrecta	Apuntar a backend público y compilar de nuevo
Variables nuevas no hacen efecto	Despliegue anterior en ejecución	Lanzar Redeploy correspondiente

15. Evidencias y entregable para Ingeniería de Software I

- Captura de GitHub con ambas carpetas y fecha/commit.
- Captura de Supabase con las tablas pets, vaccines y políticas RLS habilitadas (sin datos privados).
- Captura de Vercel de ambos proyectos, mostrando Root Directory, estado Ready y URLs, sin revelar secretos.
- Respuesta JSON de /api/health y respuesta protegida 401 sin JWT.
- Demostración de CRUD, relación mascotas–vacunas y persistencia.
- Registro de al menos una prueba de validación negativa y una prueba con usuario distinto.
- Registro de los problemas detectados y acciones correctivas realizadas.

16. Buenas prácticas y límites

Esta aplicación es un prototipo didáctico. Está preparada para aprendizaje de arquitectura, APIs, control de versiones, despliegue, autenticación y pruebas. No debe tratarse como un servicio clínico, veterinario o productivo real. Para un entorno real se necesitarían copias de seguridad, controles de abuso y rate limiting, monitoreo, revisión de dependencias, protección contra ataques y un plan de operación.

17. Referencias oficiales verificadas

- Vercel — Monorepos: https://vercel.com/docs/monorepos
- Vercel — Node.js Functions: https://vercel.com/docs/functions/runtimes/node-js
- Supabase — API keys: https://supabase.com/docs/guides/getting-started/api-keys
- Supabase — Auth redirect URLs: https://supabase.com/docs/guides/auth/redirect-urls
- Supabase — Email authentication: https://supabase.com/docs/guides/auth/general-configuration